Extracting Data from PDFs

Week 9 · Documents module · Textbook Chapter 9

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 12, 14 & 16, 2026

This week at a glance

PDFs are the web's filing cabinets. This week we pry official records out of them and turn locked documents into data.

Monday | Concepts & notebook intro

- Why PDFs resist extraction; reconstruction vs. reading
- Text layers and OCR; getting text out by hand, from readers, office tools, and the command line
- In class: the city's revenue report, by hand (20 minutes)

Wednesday | Notebook lab

- Work the Chapter 9 notebook: PyPDF, regex cleanup, Boulder City Council minutes

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 9

Course overview

Module	Week	Dates	Topics
Foundations	1	Aug. 21	Introduction & setup
	2	Aug. 24–28	Ethics, law & responsible collection
	3	Aug. 31–Sep. 4	The post-API age
	4	Sep. 9–11	Data formats: XML & JSON
	5	Sep. 14–18	Web architecture & protocols
Documents	6	Sep. 21–25	Parsing static web pages
	7	Sep. 28–Oct. 2	Archived web pages
	8	Oct. 5–9	Dynamic web pages
	9	Oct. 12–16	PDFs
APIs	10	Oct. 19–23	Wikipedia APIs
	11	Oct. 26–30	Government data APIs
	12	Nov. 2–6	Social media APIs
	13	Nov. 9–13	AI APIs
Practice	14	Nov. 16–20	Automating data collection
	15	Nov. 23–27	**No class: Fall Break**
	16	Nov. 30–Dec. 4	Final presentations
		Final Project due Monday, December 7	

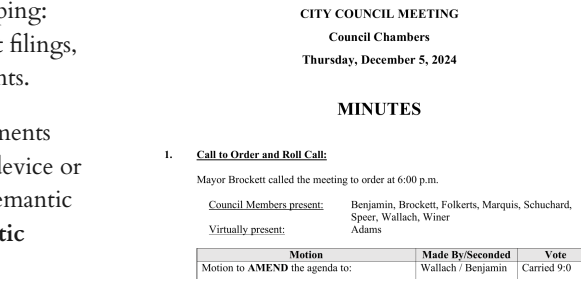
1. Monday · Concepts

The web's filing cabinets

PDFs are everywhere in institutional record-keeping: council minutes, bill texts, financial reports, court filings, academic papers, environmental impact assessments.

They are the format of choice for "official" documents because they **preserve visual layout** across any device or printer. But that priority — visual fidelity over semantic structure — makes them **hostile to programmatic extraction**.

The records inside are *readymade* data (Salganik (2018)). Somebody made them for administration, not for research. With effort you can still use them to answer questions that their authors never expected.



CITY COUNCIL MEETING

Council Chambers

Thursday, December 5, 2024

MINUTES

1. Call to Order and Roll Call:

Mayor Brockett called the meeting to order at 6:00 p.m.

Council Members present: Benjamin, Brockett, Folkerts, Marquis, Schuchard, Speer, Wallach, Winer

Virtually present: Adams

Motion	Made By/Seconded	Vote
Motion to **AMEND** the agenda to:	Wallach / Benjamin	Carried 9:0

Boulder City Council minutes, December 5, 2024: the roll call on page 1.

A computer never reads a PDF

When **you** read a PDF you see paragraphs, tables, headers, and page numbers.

A **computer** reads a different thing. It sees a sequence of drawing instructions: *put this character at* (x, y). *Draw a line from here to there. Add this image.*

So text extraction is **reconstruction, not reading** — the computer must infer logical structure from spatial layout. The results will not be perfect, but they can be good enough to turn inaccessible records into analyzable datasets.

Sales and Use Tax Summary

	December YTD				
Sources - Related to Current Economic Activity	**2023**	**2024**	**$ Change**	**YTD % Change**	**12 Month % Change**
Sales Tax	$137,732,845	$139,089,840	$1,356,995	0.97%	0.97%
Rec MJ Additional 3.5%*	1,386,752	1,184,018	-202,734	-14.62%	-14.62%
Business Use Tax	10,292,756	9,855,472	-437,285	-4.25%	-4.25%
Construction Use Tax	16,541,587	14,930,902	-1,610,685	-9.74%	-9.74%
Motor Vehicle Use Tax	6,447,628	5,965,968	-481,660	-7.47%	-7.47%
Total Sales & Use Tax based on current economic activity	**$172,401,668**	**$171,006,199**	**-$1,395,469**	**-0.81%**	**-0.81%**
Sources - Related to Prior Periods					
Sales Tax Audit Revenue	1,616,518	2,259,170	642,653	39.76%	39.76%
Use Tax Audit Revenue	1,498,943	643,062	-855,881	-57.10%	-57.10%
Total Sales and Use Tax	**$175,517,129**	**$173,908,431**	**-$1,608,698**	**-0.92%**	**-0.92%**

Boulder's December 2024 revenue report, as pdfplumber sees it: each character placed on its own (blue boxes, two rows shown), and the table's lines are 290 thin rectangles (red).

Words for getting text out

- **Text layer** — the characters a PDF stores, each with a font and a position. Select it, search it, copy it.
- **Born-digital PDF** — saved from Word, Excel, or a web page. It has a text layer.
- **Scanned (image-only) PDF** — photographs of paper pages. No text layer: nothing to select.
- **OCR**, optical character recognition — software that recognizes letters in an image and writes them out as text. **Tesseract** is the free, open-source engine most tools use.
- **Searchable PDF** — the page image with an invisible OCR text layer on top. `ocrmypdf` makes one.
- **Layout analysis** — working out columns, reading order, headers, and footers from where text sits.
- **Table extraction** — rebuilding rows and columns from lines and positions. Tabula, `pdfplumber`, `camelot`.
- **Handwriting recognition** (HTR) — OCR for handwriting. Much less accurate than print.
- **Document AI** — a vision-language model reads the page image and writes text or a table. Fast and fluent, and it can invent what it can't read: check it against the page.

First, by hand: readers and office tools

Try this before you write code. One table, once, rarely needs Python.

- **Is there text?** Drag across a line, or search with Ctrl+F (Cmd+F). If nothing highlights, it's a scan: go to OCR.
- **Copy and paste** from any reader. Paragraphs come through. A table arrives as runs of text that you split into columns yourself.
- **Adobe Acrobat Reader** (free): *File* → *Save As Other* → *Text*. Export to Word or Excel needs a paid Acrobat subscription.
- **Microsoft Word**: *File* → *Open* a PDF, and Word converts it to an editable document. Best on PDFs that are mostly text.
- **Microsoft Excel** (Microsoft 365, Windows): *Data* → *Get Data* → *From File* → *From PDF* lists the tables it finds. Not in Excel for Mac.
- **Google Drive**: right-click → *Open with* → *Google Docs* runs OCR on a PDF or image of 2 MB or less. Tables and columns are usually lost.

The command line, and free desktop tools

```
pdfinfo report.pdf # pages, producer, dates
pdffonts report.pdf # no fonts: no text layer
pdftotext -layout report.pdf report.txt
pdftoppm -png -r 200 -f 15 -l 15 report.pdf page
tesseract page-15.png page-15 # OCR one image
ocrmypdf scan.pdf searchable.pdf
```

In a notebook, start each line with `!`, as in Chapter 5.

`pdfinfo`, `pdffonts`, `pdftotext`, and `pdftoppm` come with **Poppler**: `conda install -c conda-forge poppler`. `-layout` keeps columns lined up with spaces.

`ocrmypdf` brings `tesseract` with it. Both are in `webdata`, from the week 9 setup update.

Tabula is a free desktop app made for journalists: draw a box around a table, and save it as CSV.

Online converters: weigh the upload

Smallpdf, iLovePDF, Adobe's online converter, and Google Drive will all turn a PDF into Word, Excel, or text in your browser.

Each one **sends your file to a company's server**. That is fine for a city's public revenue report. It is not fine for records with people's names, health or student information, or anything you promised to keep private.

Read the size limits and the terms. Keep the original PDF, and write down which tool made which file: that is your provenance.

Whatever the tool

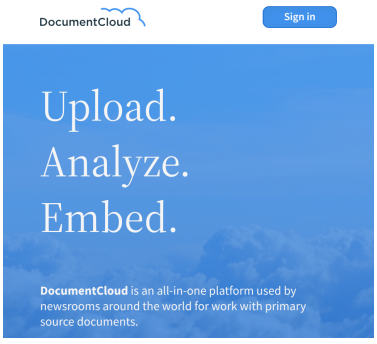
Check its output against the page. Every route here drops, merges, or misreads something, and none of them says so.

Soft enclosure & the infrastructure of transparency

Some governments publish minutes, budgets, and filings *only* as PDFs. The post-API chapter calls this **soft enclosure**. The records are public in name. In practice you cannot analyze them.

Building extraction skills enacts the **oversight** and **ownership** values — public records belong to the public, and technical barriers are themselves a form of enclosure.

Adobe created the PDF in 1993 for visual fidelity across devices — at the cost of semantic accessibility. **DocumentCloud** (ProPublica & IRE), **MuckRock** (automating FOIA), and the **Sunlight Foundation** built the infrastructure that bridges public documents and accessible data.



DocumentCloud: shared infrastructure for analyzing document dumps.

In class: the city's revenue report, by hand (20 minutes)

Work in pairs. Use no Python today.

1. Download Boulder's December 2024 revenue report from the textbook: `data/ch-09/revenue-report-2024-12.pdf`. (5 min)
2. Get the **Sales and Use Tax Summary** on page 2 into a spreadsheet. Use copy and paste. Then use one other tool from the last three slides. (8 min)
3. Clean it. Make one row per source, with the columns `source`, `2023`, and `2024`. Remove the `$` signs and the commas. (4 min)
4. Check it. Add the five sources for 2024. Compare the sum with the row "Total Sales & Use Tax based on current economic activity." (3 min)

Fast? Try the same steps on page 15, Exhibit 3.

Report back

- Which route kept the rows and columns?
- What did you fix by hand?
- Did your sum match? If not, why not?

The file

`https://github.com/cuinfoscience/Web-Data-Science-Book/tree/main/data/ch-09`

In the notebook

This week's companion notebook builds one skill at a time:

- **PyPDF basics —** open a file, read metadata, extract page-level text (and see how messy it is)
- **Cleanup —** normalize whitespace *without* destroying line structure; use regex to impose fields
- **Case study —** Boulder City Council minutes: council attendance and public-comment counts
- **Pipeline —** a reusable class that processes a whole corpus of PDFs into a DataFrame

2. Wednesday · Notebook Lab

PyPDF basics — open, inspect, extract

`pypdf` gives you document metadata and page-level text. Start by looking at what actually comes out:

```
from pypdf import PdfReader

reader = PdfReader("council_minutes_2024_01.pdf")
print(len(reader.pages), "pages")
print(reader.metadata) # title, author, creation date...

page = reader.pages[0]
text = page.extract_text()
print(text[:500]) # expect mess: headers/footers mixed in, tables garbled,
                  # line breaks that don't match paragraphs
```

The one line that flattens the page

Tempting one-liner:

```
# DON'T: \s matches newlines too,
# so this flattens the whole page
re.sub(r'\s+', ' ', raw_text)
```

Correct — collapse spaces & tabs, **keep** the line breaks:

```
text = re.sub(r'[ \t]+', ' ', raw_text)
lines = [ln.strip() for ln in text.split('\n')]
clean = '\n'.join(ln for ln in lines if ln)
```

The most important choice is what *not* to collapse.

Line structure is real information: section headings are recognizable **precisely because they sit alone on their own lines**.

Erase the newlines and no heading detector can ever find them.

Structured data hides inside a wall of characters; regex carves it out. Five patterns cover most government PDFs:

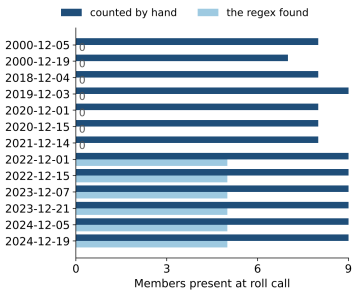
```
dates = re.findall(r'\d{1,2}/\d{1,2}/\d{2,4}', text) # 1/15/2024, 12/03/23
amounts = re.findall(r'\$[\d,]+\.?\d*', text) # $1,250.00, $3,400,000
emails = re.findall(r'[\w.+-]+@[\w-]+\.[\w.-]+', text) # clerk@bouldercolorado.gov

# ALL-CAPS section headings on their own line (a literal space, NOT \s,
# so one match can't swallow several consecutive heading lines):
headings = re.findall(r'^[A-Z][A-Z0-9 .:&-]{5,}$', text, re.MULTILINE)
# -> ['CALL TO ORDER', 'PUBLIC COMMENT', 'CONSENT AGENDA', 'ADJOURNMENT']
```

Each pattern encodes an assumption (slash dates, a leading $, uppercase headings). **Test across multiple pages and documents** before trusting a pipeline. New to regex? See *Missing Manual* Ch. 18.

Case study — council attendance

```python
def extract_attendance(pdf_path):
    text = PdfReader(pdf_path).pages[0].extract_text()
    # names follow "Present:" up to "Absent:" or end of line
    present = re.search(r'Present:\s*(.*?)\s*(?:Absent:|$)',
                        text, re.IGNORECASE | re.MULTILINE)
    absent = re.search(r'Absent:\s*(.*)$',
                       text, re.IGNORECASE | re.MULTILINE)
    return {"present": split_names(present),
            "absent": split_names(absent)}
```



Members present at roll call in 13 saved minutes: counted by hand (dark), and found by this code (light).

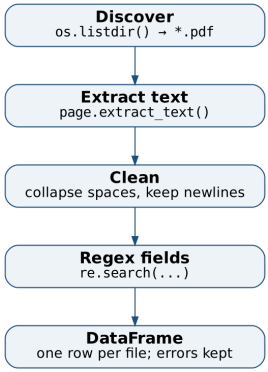
Be honest about limits: “Absent: None” becomes a phantom member named `None`; titles and separators vary. Treat output as a **first draft** — spot-check and patch.

Scale it — a reusable corpus pipeline

Wrap the workflow so one class processes a whole directory into a DataFrame:

```
class PDFCorpusAnalyzer:
    def __init__(self, directory, cleanup_patterns=None):
        self.files = sorted(f for f in os.listdir(directory)
                            if f.lower().endswith(".pdf"))
        self.cleanup_patterns = cleanup_patterns or []

    def process_all(self, extract_fn):
        rows = []
        for f in self.files:
            try: # one bad PDF won't kill the run
                text = self.extract_text(f)
                rows.append({**extract_fn(f, text), "file": f})
            except Exception as e:
                rows.append({"file": f, "error": str(e)})
        return pd.DataFrame(rows)
```



Discover → extract → clean → regex fields → DataFrame.

The limits of PyPDF

PyPDF handles text-based PDFs well, but two common cases defeat it.

Tables — PyPDF flattens them into a text stream, losing rows and columns. `pdfplumber` preserves the grid:

```
import pdfplumber
with pdfplumber.open("report.pdf") as pdf:
    table = pdf.pages[0].extract_table() # a list of row-lists (cells)
    df = pd.DataFrame(table[1:], columns=table[0])
```

Scanned PDFs — a scan has no text layer, so PyPDF returns empty or garbled output. You need **OCR**:

```
# pytesseract + the Tesseract engine turn page images into text;
# accuracy varies with scan quality. (camelot is another table option.)
```

Know your tool's limits: reach for `pdfplumber`, `camelot`, or OCR when the layout demands it.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Find one element that occurs in 10 or more government PDFs. Extract it. Put the results in a DataFrame.
2. Compare the quality of PyPDF on three documents: a text report, a document with many tables, and a slide deck.
3. Extract a budget table with `pdfplumber`. Extract it again with `pypdf`. Make numeric columns from the result.
4. Select a term such as "housing" or "police." Count it in one year of meeting minutes. Plot the spikes. Use the bag-of-words method from Ch. 7.
5. Compare `pdfplumber` and `pypdf` on the same PDF.
6. **5617**: measure the extraction quality for the PDFs of two cities. Write a memo about data quality. Use the categories of Gebru et al. (2021)

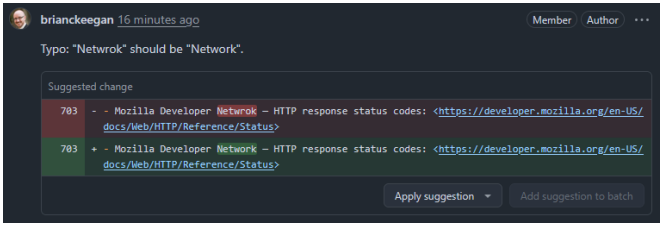
Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

3. Friday · Textbook Revisions

Improving Chapter 9 together



Today we make the book better. Each of you opens a **pull request** against `ch-09-pdfs.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

A revision menu for Chapter 9

High-value targets — pick one and scope it to a single PR:

- **Run it against real Boulder minutes.** The `# Returns:` outputs and five regex patterns are *illustrative*. Download actual signed minutes, report what the patterns catch and miss, and label sample output as representative.
- **Add the chapter's first figures.** It is all code and prose today: a "drawing-instructions vs. logical text" diagram, a pipeline flowchart, or an annotated minutes screenshot would anchor Monday's core idea.
- **Harden `extract_attendance`.** The chapter itself flags "Absent: None," title variants ("Council Member Smith"), and semicolon / line-break separators. Contribute handling **plus tests** for the formats you actually hit.
- **Make an exercise self-checking.** Add expected output, a rubric, or a starter cell to the open-ended "find and extract" and "topic tracking" exercises.
- **Deepen "When PyPDF Is Not Enough."** Turn the two-line `pdfplumber` / OCR sketches into a runnable table-extraction or Tesseract example tied to "Further Reading."

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: a new module on **APIs** — when data arrives as JSON and no parsing is required (starting with Wikipedia).

Key takeaways

1. PDFs optimize **visual fidelity over structure —** extraction is reconstruction, not reading.
2. `pypdf` gives page-level text & metadata; expect messy output and plan to clean it.
3. Collapse spaces/tabs but **keep newlines —** line structure is information; regex imposes the rest.
4. Abstract into a **pipeline** so one bad file doesn't kill a corpus run; test patterns across documents.
5. Know PyPDF's limits: `pdfplumber` for tables, OCR for scans. Imprecision is the price of scale.